

Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №3 по курсу

«Операционные системы»

Группа: М8О-214Б-23

Студент: Ысаев Р. К.

Преподаватель: Бахарев В.Д. (ФИИТ)

Оценка: _____

Дата: 07.12.24

Москва, 2024

Постановка задачи

Вариант 3.

Родительский процесс создает дочерний процесс. Первой строчкой пользователь в консоль родительского процесса пишет имя файла, которое будет передано при создании дочернего процесса. Родительский и дочерний процесс должны быть представлены разными программами. Родительский процесс передает команды пользователя через `pipe1`, который связан с стандартным входным потоком дочернего процесса. Дочерний процесс при необходимости передает данные в родительский процесс через `pipe2`. Результаты своей работы дочерний процесс пишет в созданный им файл. Допускается просто открыть файл и писать туда, не перенаправляя стандартный поток вывода. Пользователь вводит команды вида: «число число число<endline>». Далее эти числа передаются от родительского процесса в дочерний. Дочерний процесс производит деление первого числа, на последующие, а результат выводит в файл. Если происходит деление на 0, то тогда дочерний и родительский процесс завершают свою работу. Проверка деления на 0 должна осуществляться на стороне дочернего процесса. Числа имеют тип `int`. Количество чисел может быть произвольным.

Общий метод и алгоритм решения

Использованные системные вызовы:

1. `**`pid_t fork(void);`**` – создает дочерний процесс.
2. `**`ssize_t readlink(const char *path, char *buf, size_t bufsiz);`**` – читает символическую ссылку.
3. `**`ssize_t write(int fd, const void *buf, size_t count);`**` – записывает данные в файл.
4. `**`ssize_t read(int fd, void *buf, size_t count);`**` – читает данные из файла.
5. `**`int open(const char *path, int oflag, ...);`**` – открывает файл.
6. `**`int close(int fd);`**` – закрывает файл.
7. `**`int shm_open(const char *name, int oflag, mode_t mode);`**` – создает или открывает объект разделяемой памяти.
8. `**`int shm_unlink(const char *name);`**` – удаляет объект разделяемой памяти.
9. `**`int ftruncate(int fd, off_t length);`**` – устанавливает размер файла.
10. `**`void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);`**` – отображает файл или устройство в память.
11. `**`int munmap(void *addr, size_t length);`**` – отменяет отображение файла или устройства в память.
12. `**`sem_t *sem_open(const char *name, int oflag, mode_t mode, unsigned int value);`**` – создает или открывает семафор.
13. `**`int sem_close(sem_t *sem);`**` – закрывает семафор.
14. `**`int sem_unlink(const char *name);`**` – удаляет семафор.
15. `**`int sem_wait(sem_t *sem);`**` – ожидает, пока значение семафора не станет больше нуля, и уменьшает его.
16. `**`int sem_post(sem_t *sem);`**` – увеличивает значение семафора.
17. `**`pid_t wait(int *wstatus);`**` – ожидает завершения дочернего процесса.
18. `**`int execl(const char *path, char *const argv[]);`**` – заменяет текущий процесс новым процессом.
19. `**`void exit(int status);`**` – завершает выполнение программы.

Эти системные вызовы используются для управления процессами, файлами, разделяемой памятью и синхронизации между процессами с помощью семафоров.

Далее описываете то, что вы делали в рамках лабораторной работы, а также то, как работает ваша программа и т.д..

Код программы

```
child.c #include
<stdlib.h>
#include <unistd.h>
#include <string.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <semaphore.h>

#define SHM_NAME "/my_shared_memory"
#define SEM_PARENT_NAME "/parent_semaphore"
#define SEM_CHILD_NAME "/child_semaphore"
#define BUFFER_SIZE 1024

typedef struct {    char
buffer[BUFFER_SIZE];
size_t size; } shared_data;

int write_to_buffer(char *buffer, size_t buffer_size, const char *str) {
size_t len = strlen(str);    if (len > buffer_size) {        len =
buffer_size;    }    memcpy(buffer, str, len);    return len; }

void itoa(int num, char *str) {    int i = 0,
isNegative = 0;    if (num < 0)
{        isNegative = 1;        num = -num;
}    do {        str[i++] = (num % 10) + '0';
num /= 10;    } while (num > 0);    if
(isNegative) {        str[i++] = '-';    }
str[i] = '\0';    for (int j = 0, k = i - 1; j <
k; j++, k--) {        char temp = str[j];
str[j] = str[k];        str[k] = temp;    }
}

int format_result(int numerator, int denominator, char *result, size_t size) {
char num_str[32], denom_str[32], res_str[32];    itoa(numerator, num_str);
itoa(denominator, denom_str);    float div_result = (float)numerator /
denominator;    int int_part = (int)div_result;    itoa(int_part, res_str);
int offset = 0;    offset += write_to_buffer(result + offset, size - offset,
num_str);    offset += write_to_buffer(result + offset, size - offset, " /
");    offset += write_to_buffer(result + offset, size - offset, denom_str);
offset += write_to_buffer(result + offset, size - offset, " = ");    offset
+= write_to_buffer(result + offset, size - offset, res_str);    offset +=
write_to_buffer(result + offset, size - offset, "\n");    return offset; }

int main(int argc, char *argv[]) {    if (argc != 2) {
const char *msg = "Usage: ./child <filename>\n";
write(STDERR_FILENO, msg, strlen(msg));    exit(1);
}
```

```

    int shm_fd = shm_open(SHM_NAME, O_RDWR, 0666);    if
(shm_fd == -1) {          const char *err = "Error opening
shared memory\n";          write(STDERR_FILENO, err,
strlen(err));          exit(1);    }

    shared_data *shared = mmap(NULL, sizeof(shared_data),
PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0);

    sem_t *sem_parent = sem_open(SEM_PARENT_NAME, 0);
sem_t *sem_child = sem_open(SEM_CHILD_NAME, 0);

    int numbers[100];    int count;    int fd = open(argv[1],
O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd == -1) {          const char *err =
"Error opening file\n";
write(STDERR_FILENO, err, strlen(err));
exit(1);    }

    while (1)
{
    sem_wait(sem_child);

    if (shared->size == 0) {
break;
    }

    char line[BUFFER_SIZE];
memcpy(line, shared->buffer, shared->size);
line[shared->size] = '\0';    count = 0;

    char *token = strtok(line, " \n");
while (token != NULL && count < 100) {
numbers[count++] = atoi(token);
token = strtok(NULL, " \n");    }

    if (count < 2) {          const char *msg =
"Недостаточно чисел в строке\n";
write(STDERR_FILENO, msg, strlen(msg));
sem_post(sem_parent);          continue;    }

    int numerator = numbers[0];    for (int i = 1;
i < count; i++) {          if (numbers[i] == 0) {
const char *err = "Ошибка: деление на 0\n";
write(STDERR_FILENO, err, strlen(err));
write(fd, err, strlen(err));

sem_post(sem_parent);          close(fd);
munmap(shared, sizeof(shared_data));
close(shm_fd);
sem_close(sem_parent);
sem_close(sem_child);          exit(1);
}

    char result[128];          int length = format_result(numerator,
numbers[i], result, sizeof(result));          write(fd, result, length);
write(STDOUT_FILENO, result, length);    }

```

```

        sem_post(sem_parent);
    }

```

```

        close(fd);        munmap(shared,
sizeof(shared_data));    close(shm_fd);
sem_close(sem_parent);
sem_close(sem_child);

```

```

    return 0;
}

```

parent.c

```

#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/types.h>
#include <sys/mman.h>
#include <fcntl.h>
#include <semaphore.h>

```

```

#define SHM_NAME "/my_shared_memory"
#define SEM_PARENT_NAME "/parent_semaphore"
#define SEM_CHILD_NAME "/child_semaphore"
#define BUFFER_SIZE 1024

```

```

typedef struct {    char
buffer[BUFFER_SIZE];
size_t size; } shared_data;

```

```

int main(int argc, char *argv[]) {    if (argc != 2) {
const char *msg = "Usage: ./parent <filename>\n";
write(STDERR_FILENO, msg, strlen(msg));        exit(1);
}

```

```

    shm_unlink(SHM_NAME);    int shm_fd =
shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);    if (shm_fd
== -1) {        const char *err = "Error creating shared
memory\n";        write(STDERR_FILENO, err, strlen(err));
exit(1);    }

```

```

    ftruncate(shm_fd, sizeof(shared_data));
    shared_data *shared = mmap(NULL, sizeof(shared_data),
PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0);

```

```

    sem_unlink(SEM_PARENT_NAME);    sem_unlink(SEM_CHILD_NAME);
sem_t *sem_parent = sem_open(SEM_PARENT_NAME, O_CREAT, 0666, 0);
sem_t *sem_child = sem_open(SEM_CHILD_NAME, O_CREAT, 0666, 0);
pid_t pid = fork();

```

```

    if (pid < 0) {        const char *err = "Error
creating child process\n";        write(STDERR_FILENO,
err, strlen(err));        exit(1);    }

```

```

    if (pid == 0) {        execl("./child", "./child",
argv[1], NULL);        const char *err = "execl: No such file

```

```

or directory\n";          write(STDERR_FILENO, err,
strlen(err));          exit(1);
    } else {
        char buffer[BUFFER_SIZE];          const char *msg = "Введите команды
(формат: число число число<newline>, для
выхода нажмите Ctrl+D):\n";
write(STDOUT_FILENO, msg, strlen(msg));

        while ((shared->size = read(STDIN_FILENO, buffer, sizeof(buffer))) > 0)
        {
            memcpy(shared->buffer, buffer, shared->size);
sem_post(sem_child);          sem_wait(sem_parent);          }

        shared->size = 0;
sem_post(sem_child);

        munmap(shared, sizeof(shared_data));
close(shm_fd);          shm_unlink(SHM_NAME);
sem_close(sem_parent);
sem_close(sem_child);
sem_unlink(SEM_PARENT_NAME);
sem_unlink(SEM_CHILD_NAME);          }

    return
0; }

```

Протокол работы программы

strace

admin1@admin1-VMware-Virtual-Platform:~/OSY_LABS/

Laba3\$ strace ./parent 1.txt execve("./parent", ["/parent",

"1.txt"], 0x7ffd5c210278 /* 79

vars */) = 0

brk(NULL) = 0x60ab1eabd000

mmap(NULL, 8192, PROT_READ|PROT_WRITE,

MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =

0x73705832b000 access("/etc/ld.so.preload", R_OK) = -1

ENOENT (No such

file or directory)

openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|

O_CLOEXEC) = 3

fstat(3, {st_mode=S_IFREG|0644, st_size=64511, ...}) = 0

mmap(NULL, 64511, PROT_READ, MAP_PRIVATE, 3, 0) =

0x73705831b000

close(3) = 0

```

openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6",
O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0>\
0\1\0\0\0\220\243\2\0\0\0\0\0"... , 832) = 832 pread64(3, "\
6\0\0\0\4\0\0\0@\0\0\0\0\0\0@\0\0\0\0\0\0@\
0\0\0\0\0\0\0"... , 784, 64) = 784
fstat(3, {st_mode=S_IFREG|0755, st_size=2125328, ...}) = 0
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0@\0\0\0\0\0\0@\
0\0\0\0\0\0\0"... , 784, 64) = 784
mmap(NULL, 2170256, PROT_READ, MAP_PRIVATE|
MAP_DENYWRITE, 3, 0) = 0x737058000000
mmap(0x737058028000, 1605632, PROT_READ|
PROT_EXEC, MAP_PRIVATE|MAP_FIXED|
MAP_DENYWRITE, 3, 0x28000) = 0x737058028000
mmap(0x7370581b0000, 323584, PROT_READ,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x1b0000) = 0x7370581b0000
mmap(0x7370581ff000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x1fe000) = 0x7370581ff000
mmap(0x737058205000, 52624, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) =
0x737058205000
close(3) = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x737058318000
arch_prctl(ARCH_SET_FS, 0x737058318740) = 0
set_tid_address(0x737058318a10) = 4064
set_robust_list(0x737058318a20, 24) = 0
rseq(0x737058319060, 0x20, 0, 0x53053053) = 0
mprotect(0x7370581ff000, 16384, PROT_READ) = 0

```

[illegible]

[illegible]

```

child_tidptr=0x737058318a10) = 4065
write(1, "\
320\222\320\262\320\265\320\264\320\270\321\202\320\265 \
320\272\320\276\320\274\320\260\320\275\320\264\321\213 (\
321"..., 132Введите команды (формат: число число
число<newline>, для выхода нажмите Ctrl+D):
) = 132
read(0, 10 2 5
"10 2 5\n", 1024)          = 7
futex(0x737058328000, FUTEX_WAKE, 110 / 2 = 5
) = 1
10 / 5 = 2
read(0, "", 1024)          = 0
futex(0x737058328000, FUTEX_WAKE, 1) = 1
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=4065,
si_uid=1000, si_status=0, si_etime=0, si_stime=0}
--munmap(0x73705832a000, 1032) =
0
close(3)                    = 0
unlink("/dev/shm/my_shared_memory") = 0
munmap(0x737058329000, 32) = 0
munmap(0x737058328000, 32) = 0
unlink("/dev/shm/sem.parent_semaphore") = 0
unlink("/dev/shm/sem.child_semaphore") = 0
exit_group(0)               = ?
+++ exited with 0 +++

```

Вывод

Программа демонстрирует взаимодействие между родительским и дочерним процессами с использованием разделяемой памяти и семафоров для синхронизации. Родительский процесс считывает строки от пользователя и передает их дочернему процессу через разделяемую память. Дочерний процесс обрабатывает строки и записывает их в файл, а также выводит соответствующие сообщения на экран.